Relátorio AV1 | Big Data

Equipe: Cristina Matsunaga
Carlos Eduardo
Maria Fernanda Marques
Lucas Gabriel
Gabriel Chaves
Francisco da Luz
Thiago Araújo

1- Visão geral do projeto

Diante da onipresença dos cartões de crédito no dia a dia para a realização de pagamentos, o número de transações financeiras digitais aumentou exponencialmente. Esse aumento envolve tanto transações legítimas quanto tentativas de golpes e práticas fraudulentas. Isso evidencia a necessidade de detectar rapidamente esse grande volume de dados de forma quase instantânea, identificando comportamentos suspeitos para reduzir danos e prejuízos financeiros.

A frente desse contexto, esse projeto tem como objetivo o desenvolvimento de uma prova de conceito para detecção de transações fraudulentas em tempo real. Para isso, será utilizado o dataset https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud, composto por dados anônimos e amplamente utilizado em estudos de caso sobre o tema. As evidencias dessa implementação e instruções para reprodução podem ser encontrados no github: https://github.com/Carlos3du/fraud-detection-bigdata

2- Arquitetura implementada

A arquitetura do projeto foi implementada em ambiente local, com os serviços executados via Docker Compose. A solução foi organizada em três etapas principais, a de ingestão, a de armazenamento e a de transformação dos dados.

A ingestão foi simulada, pois ao invés de utilizar transações reais em tempo real, o projeto utiliza o dataset Credit Card Fraud, obtido no Kaggle. Esse arquivo CSV é lido localmente e seus registros são enviados em streaming, um a um, para um tópico no Apache Kafka, simulando a chegada contínua de transações financeiras.

No armazenamento, foi utilizado o MinIO. Embora o dataset original fique salvo localmente em CSV, os dados recebidos e processados pelo consumidor são direcionados para o MinIO, que funciona como a camada de armazenamento da solução.

A transformação foi feita com Pandas no consumidor Kafka, também em ambiente local. Nessa etapa, os dados passam por validação, conversão de tipos e criação de novos atributos, como categorização do valor da transação e agrupamento por faixa de horário. Após esse processamento, o sistema gera arquivos em formato Parquet a cada 24 segundos. Esse intervalo de tempo foi adotado para simular uma janela de 1 dia de transações.

De forma geral, a arquitetura foi implementada da seguinte forma, ingestão simulada em streaming com Kafka, armazenamento no MinIO e transformação com Pandas, com geração periódica de arquivos Parquet em ambiente local.

3- Diagrama da arquitetura

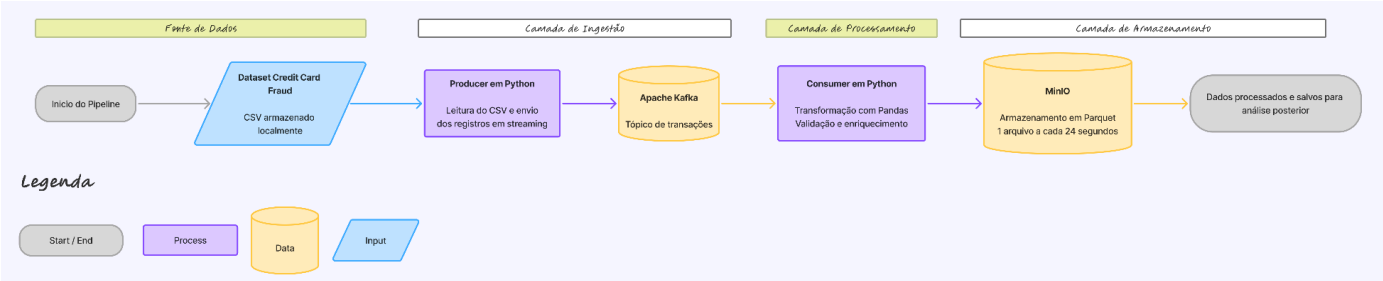

Imagem 1 - Diagrama da Arquitetura

O pipeline se inicia com a leitura do dataset de fraudes, obtido no Kaggle e armazenado localmente em CSV. Em seguida, o producer envia os registros em streaming para o Kafka. Depois disso, o consumer consome essas mensagens e realiza a transformação dos dados com Pandas. Por fim, os dados processados são armazenados no MinIO em formato Parquet, com geração de um arquivo a cada 24 segundos, encerrando o fluxo da etapa implementada.

4- Tecnologias utilizadas

Para esse trabalho foram selecionadas as seguintes tecnologias para viabilizar a prova de conceito de detecção de fraude em cartões de crédito:

- Apache Kafka → ingestão de dados
- Python / Pandas → transformação de dados
- MinIO (S3) → armazenamento

Abaixo serão apresentadas as justificativas para utilização dessas ferramentas considerando o cenário de prova de conceito, mas também serão mencionadas tecnologias alternativas muito difundidas em ambientes de produção.

Ingestão de dados: Apache Kafka

O **Apache Kafka** foi escolhido como ferramenta para ingestão de dados em tempo real, pois permite o processamento contínuo de grandes volumes de dados além da possibilidade de escalabilidade e tolerância à falha. É uma tecnologia que pode ser usada na prova de conceito pois é acessível financeiramente e que se adequa ao contexto real do nosso problema onde o volume de dados não é constante, podendo haver picos de volumes de dados exigindo velocidade e onde não pode haver perda de informação. da latência tolerável e do volume de dados:

Outra alternativa é o **Apache Pulsar**, que possui funcionalidades semelhantes ao Kafka, oferecendo melhor separação entre armazenamento e processamento, além de facilitar a escalabilidade.Também é possível utilizar soluções gerenciadas na nuvem, como o **AWS Kinesis** ou o **Google Pub/Sub**, que oferecem ingestão de dados em tempo real sem a necessidade de gerenciar infraestrutura, sendo mais indicadas para ambientes em produção.

Transformação de dados: Python e Pandas

O **Python** foi escolhido pois é a linguagem mais comumente utilizada atualmente na área de ciência de dados com possibilidade de integração com várias ferramentas. Para a etapa de transformação e tratamento de dados foi escolhido **Pandas** pois é uma ferramenta que lida bem com dados estruturados, além de ser simples de implementar e exigir menos infraestrutura. A nível de prova de conceito, o Pandas é suficiente pois dataset utilizado de transações de cartão de crédito fraudulentas não possui um grande volume de dados. No entanto, o Pandas possui limitações relacionadas ao processamento de grandes volumes, pois opera em memória e não suporta processamento distribuído.

Dessa forma, em um cenários com volume elevado de dados, outras alternativas incluem o uso do **Polars**, que apresenta melhor performance em comparação ao Pandas em operações locais e de frameworks como **Dask**, que seria uma evolução do pandas, que permite paralelismo em Python. Já num cenário de big data, onde o processamento distribuído e escalável é necessário seria mais adequado utilizar ferramentas como o **Databricks** é uma plataforma gerenciada baseada em Spark, que oferece um ambiente mais completo, com otimizações de performance. Também existe o **Amazon EMR** que permite processamento distribuído e escalável executando frameworks como Spark e Hadoop em clusters na nuvem, sem a necessidade de configurar a infraestrutura manualmente. No ambiente de cloud da Microsoft existe o **Azure Synapse Analytics** que também oferece capacidades de processamento de dados, combinando análise de dados, data warehouse e processamento distribuído em um único serviço. Em paralelo à robustez dessas ferramentas, elas exigem uma infraestrutura em nuvem paga ou funciona sistema *pay-as-you-go*.

Armazenamento: MinIO (S3)

Para o armazenamento dos dados optou-se pelo **MinIO** que é uma ferramenta open source que armazena objetos, compatível com a API do S3 e que pode simular um data lake

localmente, dispensando conexão com nuvem e também os custos agregados a infraestrutura. Para a prova de conceito é vantajoso pois é possível testar uma arquitetura próxima ao S3 dos ambientes de produção facilitando os testes e validação.

Contudo, em um cenário de Big Data em produção seria mais adequado utilizar o **Amazon S3**, **Google Cloud Storage**, **Azure Data Lake Storage** que oferecem mais robustez e escalabilidade e possuem modelo de cobrança com base no volume de armazenamento e quantidade de requisições realizadas.

5- Equipe responsável e divisão de tarefas

Divisão de Tarefas

	Cristina	Fernanda	Francisco	Lucas	Carlos	Thiago	Gabriel
Ingestão - Kafka				X			X
Armazenamento - MinIO			X				
Transformação de dados					X	X	X
Documentação / pesquisa	X	X					

6- Considerações finais

O projeto permitiu implementar uma prova de conceito funcional para detecção de transações suspeitas em um contexto de Big Data, mesmo em ambiente local e simulado. Até então, já foi estruturado um pipeline capaz de realizar a ingestão dos dados em streaming por meio do Apache Kafka, utilizando como fonte o dataset Credit Card Fraud. Também foi implementada a etapa de transformação com Python e Pandas, na qual os registros passam por validação, padronização e enriquecimento com novos atributos. Após esse processamento, os dados são armazenados no MinIO em formato Parquet, com simulação das janelas diárias de transações. Dessa forma, o projeto contempla os principais blocos da arquitetura proposta, ingestão, processamento e armazenamento.

Apesar dos avanços, a solução ainda possui limitações importantes. Por se tratar de uma implementação parcial e local, o projeto ainda não representa um ambiente de produção real, com alta disponibilidade, escalabilidade distribuída e monitoramento robusto. Além disso, a transformação dos dados foi realizada com Pandas, o que atende bem ao volume atual da prova de conceito, mas não seria a melhor alternativa para grandes volumes de dados em cenários reais de Big Data. Outro ponto a evoluir é a de uma camada analítica ou preditiva mais completa para classificação automática das transações fraudulentas em tempo real.

Como evolução futura, o projeto pode avançar para uma arquitetura mais próxima de um ambiente produtivo, com uso de ferramentas distribuídas e serviços gerenciados em nuvem. Entre as melhorias possíveis estão a substituição do Pandas por tecnologias mais escaláveis,

como Spark ou Dask, a adoção de armazenamento em nuvem como Amazon S3 ou Azure Data Lake, e a inclusão de modelos de machine learning para predição de fraude em tempo real. Também seria interessante implementar dashboards de monitoramento, alertas automáticos e métricas operacionais, ampliando o valor da solução. Assim, a prova de conceito desenvolvida estabelece uma base para futuros desenvolvimentos.

Checklist Preenchido

Formulário com o estado atual de cada parte do pipeline:

	Em Progresso	Finalizado	Pendente
Ingestão		X	
Armazenamento		X	
Transformação		X	